

SERVIÇO PÚBLICO FEDERAL · MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE VIÇOSA · UFV
CAMPUS FLORESTAL



PROVA DE CONCEITO

CLARISSON HENRIQUE DE PAIVA BENTO - 4005

Florestal - MG

Setembro de 2025

Introdução

Este documento apresenta a arquitetura proposta para o desenvolvimento do jogo **Bit Tint**, projeto do grupo do quarto ano. O objetivo principal do jogo é introduzir conceitos básicos de *bitmap* e codificação digital de cores de forma lúdica e interativa.

A mecânica central consiste em colorir uma malha quadriculada, em que cada quadrado da malha representa um *pixel*. Além disso, o jogo conta com um misturador de cores baseado no modelo *RGB*, que permite ao jogador formar novas cores a partir da combinação dos canais vermelho, verde e azul. Durante a experiência, minigames podem surgir aleatoriamente enquanto a malha é preenchida, oferecendo bônus ao jogador. Ao final da partida, é exibido um *ranking* com a pontuação dos jogadores de uma determinada turma, em formato semelhante aos tradicionais placares de arcades.

A linguagem para o *back-end* e o banco de dados foram previamente definidos como Java e MySQL, respectivamente. A partir desses requisitos, foram selecionadas as tecnologias, ferramentas, *frameworks* e bibliotecas mais adequadas para compor a solução. Um protótipo funcional foi desenvolvido como prova de conceito, com o intuito de validar a viabilidade técnica e a robustez das escolhas realizadas. Este documento descreve em detalhes o processo de construção dessa prova de conceito.

No *front-end*, optou-se pelo *framework* React, em conjunto com a ferramenta padrão de *build* Webpack. Essa escolha se deve à ampla comunidade de suporte, vasta documentação e ecossistema consolidado de bibliotecas, fatores que aceleram o desenvolvimento e reduzem a curva de aprendizado. O *front-end* foi desenvolvido em JavaScript, linguagem nativa dos navegadores e amplamente difundida, o que facilita tanto a integração com React quanto a compreensão por novos desenvolvedores. Para a navegação entre telas foi utilizada a biblioteca React Router Dom, que permite estruturar fluxos distintos dentro da aplicação de forma simples e organizada. A comunicação entre *front-end* e *back-end* foi implementada utilizando a API nativa *fetch* do JavaScript, dispensando bibliotecas externas. A estilização da interface foi realizada com CSS puro, aplicado por meio de estilos globais e *inline*.

No *back-end*, foi utilizado o Spring Boot, devido à sua robustez, facilidade de configuração e forte integração com o ecossistema Spring, especialmente no desenvolvimento de APIs RESTful. Para o gerenciamento do ciclo de vida da aplicação e suas dependências, foi adotado o Maven, garantindo padronização e facilidade de manutenção. Entre as dependências do projeto destacam-se:

- Spring Web, que fornece suporte à criação de *endpoints* REST e controle de rotas;

- Spring Boot DevTools, que agiliza o processo de desenvolvimento ao permitir reinicializações automáticas da aplicação;
- MySQL Driver, necessário para a conexão entre a aplicação e o banco de dados;
- Spring Data JPA, que abstrai operações de persistência e facilita a interação com o banco.

A arquitetura proposta promove uma separação clara entre *front-end* e *back-end*, que se comunicam exclusivamente por meio de API RESTful. Essa abordagem favorece a escalabilidade, a independência entre camadas e a possibilidade de evolução futura, permitindo, por exemplo, a substituição de um dos componentes sem comprometer os demais.

Definição Técnica

Para o gerenciamento de dependências, tanto o Spring Boot quanto o React oferecem mecanismos automatizados que simplificam a instalação de bibliotecas essenciais ao desenvolvimento. No entanto, é fundamental avaliar cuidadosamente cada nova dependência, considerando seu impacto em termos de desempenho, compatibilidade e consumo de recursos.

No *back-end*, foi utilizado o Spring Boot, com o Maven como ferramenta de *build* e gerenciamento de dependências. O Maven permite organizar o ciclo de vida do projeto Java, facilitando a configuração e garantindo reprodutibilidade do ambiente. Além disso, foram incorporadas dependências fundamentais como Spring Web, para exposição de APIs RESTful, Spring Data JPA, para abstração de acesso ao banco de dados, MySQL Driver, necessário para a conexão com o SGBD, e Spring Boot DevTools, que otimiza o fluxo de desenvolvimento com reinicializações automáticas da aplicação. A comunicação com o banco de dados ocorre via JPA/Hibernate, abstraindo comandos SQL manuais, mas ainda permitindo flexibilidade em consultas personalizadas quando necessário.

No *front-end*, optou-se pelo React, framework amplamente consolidado, que adota uma abordagem modular baseada em componentes reutilizáveis, favorecendo a escalabilidade e a manutenção do sistema. A aplicação foi estruturada com o comando “create-react-app”, que utiliza Webpack e Babel como ferramentas de *build* e *transpilation* padrões, garantindo compatibilidade com diferentes navegadores. Para a navegação entre telas, foi adotado o React Router DOM, possibilitando a implementação de fluxos distintos no jogo (*nickname*, *mixer*, *board* e *ranking*). A comunicação com o *back-end* é realizada por meio da API nativa

fetch do JavaScript, dispensando bibliotecas externas. A estilização foi desenvolvida em CSS puro, combinando regras globais e estilos *inline*.

Durante o desenvolvimento, foram utilizadas duas IDEs principais. No *back-end*, o IntelliJ IDEA foi escolhido devido à sua forte integração com Java e Spring Boot, além de recursos avançados de refatoração e depuração. No *front-end*, o Visual Studio Code foi empregado pela leveza, integração nativa com o ecossistema JavaScript e suporte a extensões que aumentam a produtividade no desenvolvimento com React.

A organização do código seguiu práticas recomendadas pelas comunidades das tecnologias envolvidas. No *front-end*, os componentes foram divididos em telas (*NicknameScreen*, *ColorMixerScreen*, *BoardScreen*, *RankingScreen*) e componentes auxiliares, promovendo clareza e separação de responsabilidades. No *back-end*, a estrutura padrão do Spring Boot foi mantida, garantindo uma arquitetura limpa e aderente às boas práticas do ecossistema. Essa divisão clara entre as camadas *front-end*, *back-end* e banco de dados promove independência, escalabilidade e facilidade de manutenção, além de reduzir a curva de aprendizado para os desenvolvedores que não são familiarizados com essas ferramentas.

Ferramentas e Instalações

As versões das ferramentas utilizadas para o desenvolvimento da prova de conceito foram as seguintes:

1. Tecnologias do Front-end:

- Node: v22.19.0
- Npm: 10.9.3
- React: 19.1.1
- React Router DOM: 7.8.2

2. Tecnologias do Back-end:

- Java: 21
- Maven: 3.8.7
- Spring Boot: 3.5.5
- Spring Web: 3.5.5
- MySQL Driver: 9.4.0
- Spring Data JPA: 3.5.5

Como criar e configurar um projeto Spring Boot

1. Acesse o Spring Initializr: <https://start.spring.io/>

2. Configure:

- Build Tool: Maven
- Linguagem: Java
- Versão do Spring Boot: 3.5.5
- Versão do Java: 21
- Packaging: Jar

3. Adicione as dependências:

- Spring Boot DevTools
- Spring Web
- MySQL Driver
- Spring Data JPA

Como criar e configurar um projeto React

O React depende do Node.js e do npm (*Node Package Manager*) para funcionar. Para obter acesse <https://nodejs.org>, baixe e instale a versão que foi indicada anteriormente.

Confirme se a instalação deu certo, abrindo um terminal e rodando:

- `node -v` (mostra a versão do Node)
- `npm -v` (mostra a versão do npm)

Com o Node instalado, crie o projeto usando o comando:

- `npx create-react-app meu-projeto`

O npx já vem junto com o npm e permite executar pacotes sem instalar globalmente. Isso cria a pasta meu-projeto com a estrutura inicial de um app React. Entre na pasta do projeto com:

- `cd meu-projeto`

Para rodar a aplicação e conferir se o projeto foi criado corretamente use:

- `npm start`

O React vai abrir no navegador em `http://localhost:3000` (por padrão).

Agora, para trabalhar com rotas (navegação entre páginas), instale o React Router DOM:

- `npm install react-router-dom`

Rode novamente o projeto para ter certeza que nenhum conflito aconteceu, se tudo correr bem, você já pode começar a programar o *front-end* da aplicação.

Links úteis

Aqui estão alguns *links* que poderão auxiliar durante o desenvolvimento do projeto, entretanto recomenda-se fortemente aos desenvolvedores procurar por outros cursos, tutoriais, métodos ou guias.

- Spring Initializr: <https://start.spring.io/>
- React: <https://pt-br.react.dev/learn>
- React Router: <https://reactrouter.com/>
- Criando Aplicação Fullstack do Zero com Java Spring e React - Parte 1: Desenvolvimento do Backend: <https://youtu.be/lUVureR5GqI>
- CURSO DE SPRING para INICIANTES | Tutorial Completo de Java Spring: https://youtu.be/YY_hf0FOIcU
- IDE Visual Studio Code: <https://code.visualstudio.com/>
- IDE IntelliJ Idea: <https://www.jetbrains.com/idea/>